

AlphaZero: Mastering Chess and Shogi by Self-Play Reinforcement Learning

Technical Summary AlphaZero Algorithm and MCTS Lineage

Abstract

AlphaZero (Silver et al., 2017) combines a single policy-value network with a PUCT-guided variant of Monte-Carlo Tree Search (MCTS). Trained by tabula rasa through self-play alone, it reached superhuman performance in chess, shogi, and Go within 24 hours, defeating Stockfish, Elmo, and the previously published AlphaGo Zero.

1. Background: Evolution of Monte-Carlo Methods

- Pure Monte-Carlo evaluation : score a position by averaging outcomes of random playouts; no tree bias toward promising lines.
- UCT (Kocsis & Szepesvári) : bandit-style tree selection, $Q(s, a) + c\sqrt{(\ln N(s))/N(s, a)}$; balances exploitation vs. exploration but forces every move to be visited once ($N = 0 \rightarrow \infty$ bonus).
- AlphaGo : adds trained policy/value networks on top of MCTS; still uses rollouts and supervised learning from human games.
- AlphaGo Zero : removes human data and rollouts; learns tabula rasa via self-play; retains Go-specific choices (symmetry augmentation, binary outcome, best-player evaluation gate).
- AlphaZero : generalises the algorithm across games: PUCT replaces UCT, a value network replaces rollouts, and all Go-specific assumptions are removed.

2. AlphaZero Architecture

2.1 Policy-Value Network

- Single deep network $(p, v) = f\theta(s)$, parameters θ .
- p: move-probability vector, $pa = Pr(a | s)$ for each action a.
- v: scalar in $[-1, +1]$, $v \approx E[z | s]$ the expected game outcome.
- Same architecture and hyperparameters reused across chess, shogi, and Go; only input/output plane encoding differs.

2.2 MCTS Four Phases per Iteration

- Selection : traverse the tree via PUCT (section 2.3), balancing exploitation against prior-weighted exploration.
- Expansion : instantiate the leaf's children; network policy p supplies each child's prior $P(s, a)$.
- Evaluation (replaces random rollout) : one network forward pass at the leaf yields $v(s)$ directly; no simulation to a terminal state.
- Backpropagation : $v(s)$ updates N and Q at every ancestor node; sign flips at each level (negamax) so Q always reflects the mover's perspective.
- Root visit counts yield a move distribution π (proportional or greedy). Illegal moves are masked to zero and probabilities renormalised; the network itself stays rule-blind.

2.3 PUCT Selection Rule

Classical UCT:

$$UCT(s, a) = Q(s, a) + c \cdot \sqrt{(\ln N(s) / N(s, a))}$$

AlphaZero's PUCT:

$$PUCT(s, a) = Q(s, a) + c \cdot P(s, a) \cdot \sqrt{N(s) / (1 + N(s, a))}$$

- $Q(s, a)$: mean backed-up value : favours moves that performed well.
- Exploration term scaled by prior $P(s, a)$: high-prior moves explored/exploited sooner; low-prior moves deprioritised.
- “+1” denominator: keeps the bonus finite at $N(s, a) = 0$ (vs. UCT's divide-by-zero), so the prior : not visit-count arithmetic governs which untried moves get attention. Enables deep, selective search instead of exhaustive breadth-first search.

3. Training Loop: Self-Play, Targets, Loss, Update

1. Self-play : both sides move via MCTS guided by the current network, sampling at $\sim \pi t$.
2. Outcome scoring : terminal position scored by rules: $z = -1$ loss, 0 draw, +1 win.
3. Targets : each position is paired with search-improved policy π (root visit counts) and outcome z .
4. Loss / update : gradient descent on $l = (z - v)^2 - \pi^T \log p + c\|\theta\|^2$: $(z - v)^2$ trains the value head (MSE vs. true outcome); $-\pi^T \log p$ trains the policy head (cross-entropy vs. π); $c\|\theta\|^2$ is L2 regularisation.
5. Continuous update : new parameters used immediately in the next self-play games; no evaluation match, no “best player” gate.

4. Key Differences

4.1 vs. AlphaGo Zero

- Outcome modelling: binary win probability (AGZ) vs. expected outcome $E[z|s]$, accounting for draws (AZ).
- Symmetry: AGZ uses 8-fold augmentation + random board transforms (Go is rotation/reflection-invariant); AZ uses neither (chess/shogi are asymmetric).
- Updating: AGZ gates on a “best player” requiring $\geq 55\%$ evaluation-match win rate; AZ updates one network continuously, no evaluation step.
- Hyperparameters: AGZ tunes per game via Bayesian optimization; AZ reuses one setting for all games, except exploration noise (scaled to each game's typical legal-move count).

4.2 vs. Classical Alpha-Beta Engines (Stockfish, Elmo)

- Classical engines: alpha-beta search + handcrafted linear evaluation + domain heuristics (quiescence search, null-move/futility pruning, transposition tables, opening books, tablebases).
- AlphaZero uses none of these MCTS averages subtree evaluations rather than computing an explicit minimax.
- Non-linear network evaluation carries more noise than tuned linear features; MCTS's averaging cancels this out, while alpha-beta's minimax propagates the single worst error to the root.
- AlphaZero searches ~ 3 orders of magnitude fewer positions/sec than Stockfish, yet scales better with thinking time challenging the belief that alpha-beta is inherently superior for chess/shogi.

5. Main Results

- Chess: outperformed Stockfish after 4 h (300k steps). Shogi: outperformed Elmo after < 2 h (110k steps). Go: outperformed AlphaGo Lee after 8 h (165k steps).

100-game matches, 1 min/move, from AlphaZero's perspective:

Opponent	Wins	Draws	Losses
Stockfish (chess)	28	72	0
Elmo (shogi)	90	2	8
AlphaGo Zero, 3-day (Go)	60	-	40

- Search speed: AlphaZero ~ 80 k pos/s (chess), ~ 40 k (shogi), ~ 16 k (Go) vs. Stockfish ~ 70 M pos/s, Elmo ~ 35 M pos/s AZ compensates with highly selective, network-guided search.

6. Why the Approach Generalised

- Chess/shogi rules are position-dependent and asymmetric (unlike Go's rotation/reflection symmetry) and include long-range piece interactions.
- AlphaZero encodes board state/actions as generic spatial planes matched to each game, without relying on symmetry.
- Removing every Go-specific assumption (section 4.1) leaves one algorithm, unmodified except input/output encoding, for all three games.
- AlphaZero independently discovered and played most common human openings during self-play, and beat Stockfish from each evidence of broad, non-narrow generalisation.

7. Conclusion

AlphaZero shows that a single, largely hyperparameter-free policy-value network with PUCT-guided MCTS, trained tabula rasa by self-play, matches or exceeds decades-refined alpha-beta engines across games of different rules and complexity replacing rollouts with learned value estimates and UCT's uninformed exploration with prior-weighted PUCT to make search selective rather than exhaustive.

References

- Silver, D., Hubert, T., Schrittwieser, J., et al. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv:1712.01815.
- Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017). Mastering the Game of Go Without Human Knowledge. *Nature*, 550, 354–359.
- Kocsis, L., & Szepesvári, C. (2006). Bandit Based Monte-Carlo Planning. *ECML*, 282–293.
- Shannon, C.E. (1950). Programming a Computer for Playing Chess. *Philosophical Magazine*, 41(314), 256–275.